Variables:

In [12]:
```python
#What is a variable in Python?
msg = "Python programming language is fun"
print(msg)
a = 34
b = 56.333
print(a , ",", b)
# Here 'msg' is the variable that is storing a string.
# So, variable is a storage location that stores any data type in Python.
```
```
Python programming language is fun
34 , 56.333
```

In [165…
```python
#How do you assign a value to a variable?
msg = "Python programming language is fun"
# As you can see above example the assignment operator "=" is helping variable to store some data.
```

In [16]:
```python
#Can you reassign a variable to a different data type?
msg = "Python programming language is fun"
msg = 45
print(msg)
# As you can see above example, it is re-assining a different data type into same variable,
# but when we use print function it's printing output only the new value that has been given not the old one.
# So, we can reassign a variable to a different data type but it's no use.
```
```
45
```

In [17]:
```python
#Explain variable naming conventions in Python.

# There are certain rules/conventions naming a variable in python:
# 1. All alphabets are allowed (a-z) & (A-Z) & "_"(underscore is also allowed)
# 2. It can start with "_" (underscores) & alphabets.
# 3. Numerics (0-9) are allowed but it shouldn't be start with.
# 4. Python is a case sensitive language (msg & MSG are different)
# 5. It shouldn't start with built-in keywords and functions.
# 6. Do not over-write the existing variable, this leads to over-write of value it's storing.
```

In [18]:
```python
#What is the difference between mutable and immutable variables?

# Variables are divided into two categories : 1) Mutable, 2) Immutable
# Mutable variables: In this category a variale can change it's object without changing it's memmory address/identity,
# and you can add, remove or modify the contents.
# Examples : "list", "dict", "set", "bytearray"

# Immutable variables: In this category a variable cannot change it's object, if anyway we tries to change it creates a new object.
# and, memmory address changes after re-assignment.
# Examples : "int", "float", "bool", etc,.
```

In [19]:
```python
#Write a Python program to assign your name to a variable and print it.
name = "Yaswanth"
print(name)
```
```
Yaswanth
```

In [21]:
```python
#Create three variables and assign them different data types.
num = 45
flt = 78.54
sti = "Python"
print(num, type(num))
print(flt, type(flt))
print(sti, type(sti))
```
```
45 <class 'int'>
78.54 <class 'float'>
Python <class 'str'>
```

In [24]:
```python
#How do you delete a variable in Python?
x = 45
print(x)
del x
print(x)
# Here we can use "del" function to delete the variable and it's value in python.
```
```
45
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[24], line 5
      3 print(x)
      4 del x
----> 5 print(x)

NameError: name 'x' is not defined
```

In [25]:
```python
#What happens if you try to use a variable before declaring it?

print(r)
# As you can see below error, it will through you a "NameError" if you try to print a variable before it's declaration.
```
```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[25], line 3
      1 #What happens if you try to use a variable before declaring it?
----> 3 print(r)

NameError: name 'r' is not defined
```

In [26]:
```python
#Explain the use of variables inside a function.
```

Identifiers:

In [27]:
```python
# What are valid identifiers in Python?

# There are certain rules/conventions naming a identifier in python:
# 1. All alphabets are allowed (a-z) & (A-Z) & "_"(underscore is also allowed)
# 2. It can start with "_" (underscores) & alphabets.
# 3. Numerics (0-9) are allowed but it shouldn't be start with.
# 4. Python is a case sensitive language (msg & MSG are different)
# 5. It shouldn't start with built-in keywords and functions.
# 6. Do not over-write the existing identifier, this leads to over-write of value it's storing.
```

In [29]:
```python
# Give 5 examples of valid and invalid identifiers.

# VALID:
# name

# age_25

# _value

# totalAmount

# MAX_VALUE

#----------------------------------------------------------------------------------------------------------

# INVALID:
# 25age

# my-name

# first name

# class

# @score
```
```
------------------------------------------------------------
```

In [30]:
```python
#Why can't Python keywords be used as identifiers?

# Because the python was developed like that way, they have seperate meaning of keywords, So we cann't use them as identifiers.
```

In [31]:
```python
# Is 'my-variable' a valid identifier?

# No, 'my-variable' is not valid identifier according to the rules.
```

In [32]:
```python
#Are identifiers case-sensitive in Python?
# Yes
```

In [33]:
```python
#Can identifiers start with an underscore?
# Yes
```

In [34]:
```python
#What is the purpose of using descriptive identifiers?

# Descriptive identifiers are mostly used in python because, it was developed like that way and it is easy to understand.
# Descriptive identifiers are names that clearly describe what the variable, function, or object represents.
```

In [35]:
```python
# Differentiate between identifiers and variables.

# Variables: A variable is a type of identifier, It is specifically used to store data or hold a value.
# Identifiers: An identifier is any name you give in Python — like the name of a variable, function, class, etc.
# and, It's just a label used to identify something in the code.

# So, all variables are identifiers, but not all identifiers are variables.
```

In [36]:
```python
# Is 'class' a valid identifier? Why or why not?
# class is not a valid identifier, because according to the rules.
# In python it is designed as a keyword that defines a class.
```

In [38]:
```python
# How do naming conventions affect readability in code?

#Naming conventions make code easier to read and understand.
#Good names explain what the variable or function is for.
#Consistent and clear naming helps with debugging.
```

Operators:

In [39]:
```python
# What are the different types of operators in Python?

# Arithmetic Operators - For math operations
# +, -, *, /, %, //, **

# Comparison Operators - To compare values
# ==, !=, >, <, >=, <=

# Assignment Operators - To assign values
# =, +=, -=, *=, /=, //=, %=, **=

# Logical Operators - For logic (True/False)
# and, or, not

# Bitwise Operators - For binary (bit) operations
# &, |, ^, ~, <<, >>

# Membership Operators - To check if a value is in a sequence
# in, not in

# Identity Operators - To check if two variables are the same object
# is, is not
```

In [40]:
```python
# Write a program to demonstrate arithmetic operators.
a = 45
b = 56
print(a+b)
print(a-b)
print(a*b)
print(a/b)
print(a%b)
print(a//b)
print(a**b)
```
```
101
-11
2520
0.8035714285714286
45
0
380102539714517509119234755043630811768952009743379517880956797171165817417204380035400390625
```

In [41]:
```python
# What is the result of 5 ** 3 in Python?
print(5 ** 3)
```
```
125
```

In [42]:
```python
# Explain the difference between / and // operators.
c = 34
d = 43
print(c / d)
print(c // d)
# "/" --> division operator
# Gives the exact (float) result of division.
# Output is always a decimal number.

# "//"--> floor division operator
# Gives the whole number part of the result.
# It removes the decimal and rounds down.
```
```
0.7906976744186046
0
```

In [43]:
```python
# How does the modulo operator work? Give an example.
# The modulo operator % gives the remainder of a division.
print(10 % 3)
print(15 % 5)
```
```
1
0
```

In [44]:
```python
# Demonstrate logical operators with truth tables.
values = [True, False]

print("Truth Table for AND:")
print("A\tB\tA and B")
for a in values:
    for b in values:
        print(f"{a}\t{b}\t{a and b}")

print("\nTruth Table for OR:")
print("A\tB\tA or B")
for a in values:
    for b in values:
        print(f"{a}\t{b}\t{a or b}")

print("\nTruth Table for NOT:")
print("A\tnot A")
for a in values:
    print(f"{a}\t{not a}")
```
```
Truth Table for AND:
A       B       A and B
True    True    True
True    False   False
False   True    False
False   False   False

Truth Table for OR:
A       B       A or B
True    True    True
True    False   True
False   True    True
False   False   False

Truth Table for NOT:
A       not A
True    False
False   True
```

In [45]:
```python
# Write a Python program using comparison operators.
e = 3
f = 4
print(e == f)
print(e != f)
print(e >= f)
print(e <= f)
print(e<f)
print(e>f)
```
```
False
True
False
True
True
False
```

In [52]:
```python
# Use assignment operators to update a variable.
x = 10
print("Initial x =", x)

x += 5
print("After x += 5:", x)

x -= 2
print("After x -= 2:", x)

x *= 3
print("After x *= 3:", x)

x /= 3
print("After x /= 3:", x)

x %= 4
print("After x %= 4:", x)
```
```
Initial x = 10
After x += 5: 15
After x -= 2: 13
After x *= 3: 39
After x /= 3: 13.0
After x %= 4: 1.0
```

In [56]:
```python
# What is the output of: a = 5; b = a += 2?
#a = 5
#b = a += 2
#print(a, ",", b)
# The syntax given is wrong the correct syntax is:
a = 5
a += 5
b = a
print(a , ",", b)
```
```
10 , 10
```

In [58]:
```python
# What are identity operators? Provide examples.
# Identity Operators - To check if two variables are the same object
# is, is not
# Examples:
my = "My name is Yasawnth"
my1 = "My name is yaswanth"
print(my is my1)
print(my is not my1)
```
```
False
True
```

Strings - Indexing and Slicing:

In [59]:
```python
# What is indexing in strings?
# Basically the strings in python are stored in indexed form, meaning each character has unique position and value.
```

In [61]:
```python
# Demonstrate accessing the 3rd character of a string.
ert = "Python"
print(ert[3])
```
```
h
```

In [64]:
```python
# Explain slicing with an example.
# Slicing: This allows to access the parts of the string or slice them.
# Example:
print(ert)
print(ert[0:5])
print(ert[0:4])
```
```
Python
Pytho
Pyth
```

In [72]:
```python
# What is negative indexing? Show with an example.
# Negative indexing means accessing elements from the end of a list, string, or tuple using negative numbers.
# Example:
print(ert[-5:2])
print(ert[::-1])
```
```
y
nohtyP
```

In [73]:
```python
# Write a program to print every second character in a string.
my = "My name is Yasawnth"
print(my[::2])
```
```
M aei aanh
```

In [74]:
```python
# Get a substring from index 2 to 6 of the string 'Programming'.
f = "Programming"
print(f[2:7])
```
```
ogram
```

In [75]:
```python
# Write code to reverse a string using slicing.
f = "Programming"
print(f[::-1])
```
```
gnimmargorP
```

In [76]:
```python
# What will be the result of 'hello'[1:4]?
t = "hello"
print(t[1:4])
```
```
ell
```

In [80]:
```python
# What happens when slicing indexes are out of range?
# If slicing indexes are out of range then it will give the output to the end of the string
# Example:
t = "hello"
print(t[1:10000000000])
```
```
ello
```

In [87]:
```python
# How do you get the last character of a string?
t = "hello"
print(t[-1])
# Because for any string the last character value is "-1"
```
```
o
```

String Methods:

In [89]:
```python
# Write a program to convert a string to uppercase.
t = "hello"
print(t.upper())
```
```
HELLO
```

In [91]:
```python
# Demonstrate use of .lower(), .capitalize() and .title().
t = "python is very funnn!"
print(t.lower())
print(t.capitalize())
print(t.title())
```
```
python is very funnn!
Python is very funnn!
Python Is Very Funnn!
```

In [94]:
```python
# What does .strip() do? Show an example.
r = "   hello    "
q = "*******hello*******"
print(r.strip())
print(q.strip("*"))
```
```
hello
hello
```

In [95]:
```python
# Replace 'cat' with 'dog' in the string 'The cat sat there'.
v = "The cat sat there"
print(v.replace("cat", "dog"))
```
```
The dog sat there
```

In [96]:
```python
# Use .find() to locate a substring.
v = "The cat sat there"
print(v.find("ere"))
```
```
14
```

In [98]:
```python
# How does .split() work? Give an example.
v = "The cat sat there"
print(v.split())
```
```
['The', 'cat', 'sat', 'there']
```

In [101…
```python
# Combine a list of words into a single string using .join().
words = ["Hi" , "Hello"]
print(" ".join(words))
```
```
Hi Hello
```

In [103…
```python
# What is the use of .isdigit() and .isalpha()?
z = "1234"
m = "Hi"
p = "Hi3545"
print(z.isalpha())# checks if characters are all alphabets or not
print(m.isdigit())# checks if characters are all digits or not
print(p.isalnum())# checks if all the characters are mix of alphabets(a-z & A-Z) and numbers(0-9)
```
```
False
False
True
```

In [104…
```python
# Write a program to count the number of 'a's in a string.
u = "Heloooooooooo"
print(u.count("o"))
```
```
9
```

In [108…
```python
# Demonstrate chaining multiple string methods together.
v = "the cat sat there"
print(v.capitalize().upper().split())
```
```
['THE', 'CAT', 'SAT', 'THERE']
```

f-strings and .format

In [109…
```python
# Write a program using f-string to print name and age.
name = "Yaswanth"
age = 22
print(f"My name is {name}, and I am {age} years old")
```
```
My name is Yaswanth, and I am 22 years old
```

In [110…
```python
# Demonstrate the use of .format() with placeholders.
print("My name is {}, and I am {} years old".format("Yaswanth", "22"))
```
```
My name is Yaswanth, and I am 22 years old
```

In [42]:
```python
# Compare f-strings with .format() method.
print("This is the comparision:")
price = 34.67
print(f"The total bill is: {price}$")
print("--"*45)
print("My name is {}, and I am {} years old".format("Yaswanth", "22"))
# These are basically methods of using f-string, for me using f-string is much more better than .format()
```
```
This is the comparision:
The total bill is: 34.67$
------------------------------------------------------------------------------------------
My name is Yaswanth, and I am 22 years old
```

In [113…
```python
# Format a float value to 2 decimal places using f-string.
price = 34.67
print(f"The total bill is: {price}$")
```
```
The total bill is: 34.67$
```

In [114…
```python
# Create a sentence using named placeholders with .format().
print("{} {} {} {}".format("The", "Weather", "is", "Beautiful Today"))
```
```
The Weather is Beautiful Today
```

In [116…
```python
# How do you insert variables inside a string using f-string?
price = 34.67
print(f"The total bill is: {price}$")
# First we have to declare a variable that we have to insert in a string.
# Then print the whole string using f-string as you can see in above example,
# At last you have to declare the varible name in curly braces "{}" at the right position your desired position.
```
```
The total bill is: 34.67$
```

In [122…
```python
# Align a string to the center using format syntax.
text = "Yaswanth"
print(f"{text:^21}")
# Doubt*****
```
```
      Yaswanth
```

In [123…
```python
# Display leading zeros in a number using .format().
print("{}{}{}{}{}256".format("0","0","0","0","0"))
```
```
00000256
```

In [128…
```python
# Create a multiline formatted string using f-string.
studnt, course, institute, floor = "Yaswanth", "Data Analytics", "Innomatics", 2
print(f"Hi, My name is {studnt}, I have enrolled in {course} course \n in {institute}, situated in {floor}nd floor")
```
```
Hi, My name is Yaswanth, I have enrolled in Data Analytics course
 in Innomatics, situated in 2nd floor
```

In [131…
```python
# Use f-string to embed an expression like 2+2.
print(f"The result of 4 + 6 is {4 + 6}")
```
```
The result of 4 + 6 is 10
```

Python Basics:

In [132…
```python
# List basic data types in Python.

# Basically there are 5 data types in python:
# 1) int: Integer Numbers, Ex: x = 10
# 2) float: Decimal Numbers, Ex: y = 23.56
# 3) str: Text (string), Ex: msg = "Python"
# 4) bool: Boolean values (True or False), Ex: is_valid = True
# 5) NoneType: There will be no value in this data type, Ex: value = None
```

In [133…
```python
# What is a keyword? Give 5 examples.

# Keyword : A keyword is bascially reserved words in python, these have special meanings,
# These keywords cannot be used as identifiers and variables and for functions also.
# Examples: if, for, while, else, def, etc,.
```

In [134…
```python
# Write a program to swap two numbers without a third variable.
a = 20
b = 43
a , b = b , a
print("a = ", a)
print("b = ", b)
```
```
a =  43
b =  20
```

In [135…
```python
# What is indentation and why is it important?

# Identation: Identation means adding spaces or tabs at the begining of a line to show code structure.
# Python is idented language this is important because it was developed in that way, this defines block of codees.
# functions like if, for, else, etc,. loops plays major important role of identation.
# If you write a code without identation, it raises a "IdentationError", this is why it is important.
```

In [136…
```python
# Define a function and call it in Python.

# Function: It is a small block of code that gives output without writing it every time.
# We can write it once and we can use it any where without full code.
# Example:
def sayhello():
    print("Welcome Yaswanth")
sayhello()
```
```
Welcome Yaswanth
```

In [137…
```python
# Explain the use of the input() function.

# It is used to take the input from the user while the program is executing.
# Example:
w = int(input("Enter a number: "))
print(w)
```
```
34
```

In [138…
```python
# What is the purpose of print()? How to print multiple values?

# The print function is used to display the output on the screen.
# It shows "messages, variable values, results, calculations"
# We can print multiple values by seperating each one with ","(commas)
name = "Yaswamth"
age = 22
print(name, age )
```
```
Yaswamth 22
```

In [142…
```python
# What are comments in Python? Write single and multi-line comments.

# Comments are used for readbility of the code
# They are : #, '', "", ''' ''', """ """.
# These are can be used to enclose the text/string data
# single quotes are used for single lines/text.
# Multiple quotes are used for multi-lines/multi text.

str3 = 'My name is Yaswanth'

str1 = """In programming, comments are text notes embedded in the source code
that are ignored by the compiler or interpreter during execution.
They serve as documentation and help explain the code's logic and
purpose to other developers or even the programmer themselves at a later time."""
```

In [144…
```python
# Write a program to take name and age from user and print.
user_name = input("Enter Name: ")
user_age = int(input("Enter Age: "))
```

In [145…
```python
# Create a Python file and run it from terminal or command prompt.
```

Coding Questions Related to Strings:

In [7]:
```python
# Write a program to reverse a string without using [::-1].
name = "Yaswanth"
b = "".join(reversed(name))
print(type(b))
print(b)
```
```
<class 'str'>
htnawsaY
```

In [147…
```python
# Count the number of vowels and consonants in a string.
```

In [43]:
```python
# Check if a string is a palindrome.
a = "sas"
if a == a[::-1]:
    print("Palindrome")
else:
    print("Not palindrome")
```
```
Palindrome
```

In [149…
```python
# Remove duplicate characters from a string.
```

In [150…
```python
# Find the first non-repeating character in a string.
```

In [45]:
```python
# Check if two strings are anagrams.
r = "bat"
t = "tab"
if sorted(r) == sorted(t):
    print("Anagram")
else:
    print("Not Anagram")
```
```
Anagram
```

In [152…
```python
# Write a program to print all substrings of a string.
```

In [46]:
```python
# Find the longest word in a sentence.
str4 = "My name is Yaswanth"
x = str4.split()
l = ""
for i in x:
    if len(i) > len(l):
        l = i
print(l)
```
```
Yaswanth
```

In [154…
```python
# Capitalize the first letter of every word.
str1 = """In programming, comments are text notes embedded in the source code
that are ignored by the compiler or interpreter during execution.
They serve as documentation and help explain the code's logic and
purpose to other developers or even the programmer themselves at a later time."""
print(str1.title())
```
```
In Programming, Comments Are Text Notes Embedded In The Source Code
That Are Ignored By The Compiler Or Interpreter During Execution.
They Serve As Documentation And Help Explain The Code'S Logic And
Purpose To Other Developers Or Even The Programmer Themselves At A Later Time.
```

In [155…
```python
# Count how many times a character appears in a string.
str1 = """In programming, comments are text notes embedded in the source code
that are ignored by the compiler or interpreter during execution.
They serve as documentation and help explain the code's logic and
purpose to other developers or even the programmer themselves at a later time."""
print(str1.count("m"))
```
```
11
```

Python Typecasting:

In [3]:
```python
# Convert string '123' to int and perform addition.
num_1 = "123"
num_2 = "456"
a = int(num_1)
b = int(num_2)
print(a+b)
print(type(a+b))
```
```
579
<class 'int'>
```

In [9]:
```python
# Convert integer 456 to string and concatenate with another string.
a = 456
b = str(a)
c = "789"
print(b+c)
print(type(b+c))
```
```
456789
<class 'str'>
```

In [11]:
```python
# Convert float 45.67 to int and observe the result.
flt = 45.67
e = int(flt)
print(e)
print(type(e))
```
```
45
<class 'int'>
```

In [13]:
```python
# Convert boolean True to int and False to int.
t = True
f = False
T = int(t)
F = int(f)
print(T, ",",  F)
```
```
1 , 0
```

In [14]:
```python
# Convert int 100 to float and check its type.
i1  = 100
f1 = float(i1)
print(type(f1))
```
```
<class 'float'>
```

In [17]:
```python
# Take user input and convert to integer safely.
s2 = input()
f2 = int(s2)
print(type(f2))
```
```
<class 'int'>
```

In [18]:
```python
# What is the output of str(5+3)? Explain.
print(str(5+3))
```
```
8
```

In [19]:
```python
# Convert a list of numeric strings to integers using map().
```

In [21]:
```python
# Convert string '12.5' to float and round it.
str2 = '12.5'
n = float(str2)
q = int(n)
print(q)
```
```
12
```

In [22]:
```python
# Explain implicit vs explicit type casting with examples.
```

Extra Practice Coding:

In [35]:
```python
# Find all words longer than 5 characters in a sentence.
str4 = "My name is Yaswanth"
```

In [24]:
```python
# Remove punctuation from a string.
```

In [36]:
```python
# Write a program to change case of each character in a string.
str4 = "My name is Yaswanth"
print(str4.swapcase())
```
```
mY NAME IS yASWANTH
```

In [26]:
```python
# Create a program that counts the occurrence of each word.
```

In [27]:
```python
# Extract digits from a string and form a new number.
```

In [37]:
```python
# Write a function to check if a string contains only digits.
str5 = "3445667"
print(str5.isdigit())
```
```
True
```

In [38]:
```python
# Create a function to remove vowels from a string.
str4 = "My name is Yaswanth"
print(str4.)
```
```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[38], line 3
      1 # Create a function to remove vowels from a string.
      2 str4 = "My name is Yaswanth"
----> 3 print(str4.remove("a","e","i","o","u"))

AttributeError: 'str' object has no attribute 'remove'
```

In [39]:
```python
# Write a function to check if a character is uppercase.
str4 = "My name is Yaswanth"
print(str4.isupper())
```
```
False
```

In [40]:
```python
# Replace spaces with hyphens in a string.
str4 = "My name is Yaswanth"
print(str4.replace(" ", "-"))
```
```
My-name-is-Yaswanth
```